

Generics em TypeScript

Miguel Costa
miguelangcosta@gmail.com

Objetivos

- O que são generics
- Para que servem
- Porque existem
- Como funcionam
- O que é o `<T>`
- Se é JavaScript ou TypeScript
- Como usar
- Onde usar
- O que pode e não pode fazer
- Como escrever funções genéricas
- Como escrever classes genéricas

O problema sem generics

```
function showNumber(n: number) {  
    return n;  
}  
  
function showString(s: string) {  
    return s;  
}
```

- Código repetido
- Muitas funções
- Muito trabalho
- Difícil de manter

A solução com generics

```
function show<T>(value: T) {  
  return value;  
}
```

Uso:

```
show(10);  
show("olá");  
show(true);
```

- ✓ Uma função
- ✓ Todos os tipos
- ✓ Código limpo
- ✓ Reutilização
- ✓ Segurança de tipos

O que são Generics?

Generic é código que funciona com qualquer tipo, mas continua tipado.

“Uma função/classe que funciona para todos os tipos.”

O que é o `<T>`?

```
function show<T>(value: T): T
```

Explicação:

- `<T>` cria o tipo
- `value: T` usa o tipo
- `: T` devolve o tipo

O **<T>** é obrigatório?

Não!

- Pode ser qualquer nome:
- **T** é só costume (convenção)

```
function show<Tipo>(value: Tipo): Tipo  
function show<X>(value: X): X  
function show<Coisa>(value: Coisa): Coisa
```

Isto é JavaScript?

Não!

- Isto é TypeScript
- `<T>` não existe em JavaScript
- `: T` não existe em JavaScript

JS:

```
function show(value) {  
  return value;  
}
```

TypeScript:

```
function show<T>(value: T): T {  
  return value;  
}
```


O **T** tem de estar no parâmetro e no retorno?

Não!

Só no parâmetro:

```
function log<T>(value: T): void {  
    console.log(value);  
}
```

Só no retorno:

```
function create<T>(): T {  
    return {} as T;  
}
```

Pode existir mais do que um generic?

Sim!

```
function pair<T, U>(a: T, b: U) {  
  return [a, b];  
}
```

Uso:

```
pair(1, "a");  
pair(true, 10);
```

Exemplo:

Função que cria array

```
function toArray<T>(value: T): T[] {  
    return [value];  
}
```

Uso:

```
toArray(1);  
toArray("a");
```

Classe genérica simples

```
class Box<T> {  
    value: T;  
  
    constructor(value: T) {  
        this.value = value;  
    }  
}
```

Uso:

```
const b1 = new Box(10);  
const b2 = new Box("Olá");
```

Classe com lista

```
class List<T> {  
    items: T[] = [];  
  
    add(item: T) {  
        this.items.push(item);  
    }  
  
    getAll() {  
        return this.items;  
    }  
}
```

Uso:

```
const numbers = new List<number>();  
numbers.add(1);  
  
const names = new List<string>();  
names.add("Ana");
```

Generics com interfaces

```
interface Box<T> {  
  value: T;  
}
```

Uso:

```
const box1: Box<number> = { value: 10 };  
const box2: Box<string> = { value: "Olá" };
```

Generics com classes abstratas

- Uma **classe abstrata** define regras.
- Um **generic** define tipos flexíveis.

```
abstract class Box<T> {  
    abstract get(): T;  
}
```

Box é uma classe abstrata → não pode ser instanciada

<T> é um tipo genérico → tipo ainda não é conhecido

get(): T → quem herdar **tem de devolver T**

Generics com classes abstratas

```
abstract class Box<T> {  
    abstract get(): T;  
}
```

Classe filha define o tipo

```
class NumberBox extends Box<number> {  
    get(): number {  
        return 10;  
    }  
}
```

```
class StringBox extends Box<string> {  
    get(): string {  
        return "Olá";  
    }  
}
```


Generics com classes abstratas

```
abstract class Box<T>
```

É como dizer:

“Existe uma classe chamada Box.

Ela trabalha com um tipo que eu ainda não sei qual é.

Quem herdar é que vai decidir o tipo.”

Generic no parâmetro da função abstrata

```
abstract class Storage<T> {  
    abstract save(value: T): void;  
}
```

Classe filha:

função abstrata obriga:

- a existir **save**
- a receber **number**

```
class NumberStorage extends Storage<number> {  
    save(value: number) {  
        console.log(value);  
    }  
}
```

Generic no retorno da função abstrata

```
abstract class Creator<T> {  
    abstract create(): T;  
}
```

Classe filha:

```
class StringCreator extends Creator<string> {  
    create(): string {  
        return "Olá";  
    }  
}
```

Generic em classes

É possível ter funções genéricas dentro de uma classe que não tem tipo definido?

Sim! Exemplo: classe normal (não genérica) com função genérica

```
class Utils {  
    echo<T>(value: T): T {  
        return value;  
    }  
}
```

Uso!

```
const u = new Utils();  
u.echo(10);      // number  
u.echo("abc");   // string
```

- A classe **não é genérica**
- Mas a função **é genérica**
- O **<T>** pertence só à função

Classe genérica + função genérica

Aqui existem **dois generics diferentes**:

- **T** → da classe
- **U** → da função

```
class Box<T> {  
    value: T;  
  
    constructor(value: T) {  
        this.value = value;  
    }  
  
    echo<U>(value: U): U {  
        return value;  
    }  
}
```

Resumo mental simples

Classe genérica

```
class Box<T> {}
```

Resumo mental simples

Função genérica

```
function echo<T>(v: T): T {}
```

Resumo mental simples

Classe normal + Função genérica

```
class Utils {  
    echo<T>(v: T): T {}  
}
```


Resumo mental simples

Classe genérica + Função genérica

```
class Box<T> {  
    echo<U>(v: U): U {}  
}
```

Quando usar Generics?

- ✓ código se repete
- ✓ mesma lógica para vários tipos
- ✓ reutilização
- ✓ classes reutilizáveis
- ✓ funções reutilizáveis

Boas práticas:

- ✓ começar simples
- ✓ usar poucos generics
- ✓ código claro
- ✓ exemplos simples
- ✓ não complicar

Quando NÃO usar?

- ✗ só existe um tipo
- ✗ não há repetição
- ✗ complica sem necessidade

Questões